

Computer Programming Lecture 2

Outline

- Variables
- Expressions
- Statements
- Data types

Variable

- Variable refers to a named place in the memory where a programmer can store data and later retrieve it.
- Programmers get to choose the names of the variables

```
>>> x = 3
```

```
>>> message = "Hello"
```

```
>>> print(x) -> 3
```

```
>>> print(x + 2) -> 5
```

```
>>> print(message) -> Hello
```

```
name = "Michael"  
print(name)
```

Exercise

- Computing area of a rectangle
- Computing area of a triangle

Variables

- You can change the content of a variable:

```
>>> x = 3
```

```
>>> x = 4
```

```
>>> print(x) -> 4
```

Python Variable Name Rules

- Must start with a letter or underscore _
- Must consist of letters and numbers and underscores
- Case Sensitive
- **Good:** spam eggs spam23 _speed
- **Bad:** 23spam #sign var.12
- **Different:** spam Spam SPAM

Reserved keywords

You can not use reserved keywords as identifier names

and del for is raise
assert elif from lambda return
break else global not try
class except if or while
continue exec import pass yield
def finally in

Named Constants

- They are similar to variables: a memory location that's been given a name.
- Unlike variables their contents shouldn't change.

```
>>> PI = 3.14
```

Literals

- Literals in Python are raw values or data that are assigned to variables or constants.
- There are different types of literals:
 - String literal -> "Hello", 'Hi', '''this is multi-line literal'''
 - Numeric literal -> 1, 1.5
 - Boolean literal -> True, False
 - Literal collections -> [1,2,3,4], (1,2)

Expressions

- An expression is a combination of operands and operators.
- Operators are special symbols that designate some sort of computation, such as addition, subtraction, multiplication, etc. Operands are the values that an operator acts on, such as numbers, strings, variables, etc.

```
>>> radius = 2
```

```
>>> PI = 3.14
```

```
>>> area = PI * (r ** 2)
```

```
>>> print(area)
```

Statements

- A statement is a unit of code that performs some action but does not return any value.

Example:

- Assignment Statement -> `x = 2`
- Assignment statement with an expression -> `x = 2 + 3`
- Print statement -> `print(x)`
- Expressions can be used as part of statements, but not vice versa.
 - For example, you can assign an expression to a variable (`x = 2 + 3`) or print an expression (`print(a * b)`), but you cannot use a statement as an operand (`2 + print("Hello")`)

Assignment statement

- Assignment statement assigns value to a variable.
- It has the following form:
 - `<variable> = <expression>`
- `x = 7`
- `x = x + 7 -> 14`

Numeric Expressions

Operation	Symbol
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Power
%	Remainder
//	Floor division

Operation Precedence

- Dictates order of execution in evaluating arithmetic expressions.
 - Parenthesis
 - Power
 - Multiplication and division
 - Addition and Subtraction
 - Left to right

Example

$$x = 1 + 2 ** 3 / 4 * 5$$

Evaluation:

$1 + 8 / 4 * 5$ -> Power gets evaluated

$1 + 2 * 5$ -> The division gets evaluated due to left to right rule

$1 + 10$ -> The multiplication gets evaluated

11 -> the addition gets evaluated

Exercise

```
>>> 2 + 3 - 1 * 5 + 1 / 4
```

```
>>> 2 + 3 - 1 * (5 + 1/4)
```

Data Types

- Data types are classifications of data that tell the compiler or interpreter how the programmer intends to use the data.
- Data types define the set of values and operations that can be applied on those values.

Numeric Types

- This types are for data that is numeric or should be worked with as a number for addition or other math operations.
- Numbers can be integers or floating point (numbers with fractional parts).
 - Integers are represented using int data type in python.
 - Floating point numbers are represented using float data type.
- Other numeric types:
 - Complex numbers: also supports complex numbers.

String

- String is a sequence of characters.
- `>>> name = "Abebe"`
- `>>> message = "Hello " + name`
- `>>> print(message) -> Hello Abebe`

Boolean

- We have two values in Boolean data type:
 - True
 - False

Type in Python

- You can check the type of a variable or value using type function:

```
>>> type(2) -> <class 'int'>
```

```
>>> type(2.0) -> <class 'float'>
```

```
>>> type("hello") -> <class 'str'>
```

```
>>> type(True) -> <class 'bool'>
```

Mixing Integer and Floating Point Numbers

- When you perform an operation where one operand is an integer and the other operand is a floating point the result is a floating point.
- `>>> 2 + 2.0 -> 4.0`
- `>>> 99 // 100 -> 0`
- `>>> 99.0 // 100 -> 0.0`

Type Conversions

- You can convert values explicitly between numeric types using `float()` and `int()`.
- `>>> float(4) -> 4.0`
- `>>> int(3.2) -> 3`
- If a string does not contain non-numeric characters it can also be converted to float and integer.
- `>>> float("4.2") -> 4.2`
- `>>> int("3") -> 3`
- `>>> int("3.2") -> Error`

Variable names

- Since we programmers are given a choice in how we choose our variable names.
- We name variables to help us remember what we intend to store in them.

Exercise

Using input to read input from users.

-> Exercise 1: Write a program that accepts your name and prints hello with your name in it.

-> Exercise 2: Write a program that accepts input from user for length and width and calculates area and perimeter of rectangle.

-> Exercise 3: Write a program that converts any currency to another by accepting rate and amount from the user.